

Lab 4 VibeCoding - Scenario 22 Sliding Puzzle: LLM Chat History

Student: Pranav Prabhakar · Tool: Claude Code (Claude desktop app) · Models: Claude Sonnet 5.5, then Claude Opus 5.5 (switched via /model) · Date: 7 October 2026

Complete conversation, exported from the session transcript. Tool calls and code edits are shown in full; long tool output is truncated. The assistant's internal reasoning is not included.

Paths are relative to the project folder (22_sliding_puzzle); "scratchpad/" is Claude Code's temporary working folder, which is not part of the project.

USER - PROMPT 1

Please go through the entire source code and the readme file in detail. Describe the task and objective given. Do not get working on the actual code yet.

ASSISTANT - CLAUDE SONNET 5.5 (CLAUDE CODE)

Tool call: **Glob** **/*

Result

```
.git\description
.git\hooks\applypatch-msg.sample
.git\hooks\commit-msg.sample
.git\hooks\fsmonitor-watchman.sample
.git\hooks\post-update.sample
.git\hooks\pre-applypatch.sample
.git\hooks\pre-commit.sample
.git\hooks\pre-merge-commit.sample
.git\hooks\pre-push.sample
.git\hooks\pre-rebase.sample
.git\hooks\pre-receive.sample
.git\hooks\prepare-commit-msg.sample
.git\hooks\push-to-checkout.sample
.git\hooks\sendemail-validate.sample
.git\hooks\update.sample
.git\info\exclude
.git\objects\pack\pack-b79a4558f84a55183ba9b84343cd1e4ed9acd59c.pack
.git\objects\pack\pack-b79a4558f84a55183ba9b84343cd1e4ed9acd59c.idx
.git\objects\pack\pack-b79a4558f84a55183ba9b84343cd1e4ed9acd59c.rev
.git\packed-refs
.git\refs\remotes\origin\HEAD
.git\logs\refs\remotes\origin\HEAD
.git\HEAD
.git\refs\heads\main
.git\logs\HEAD
.git\logs\refs\heads\main
README.md
game.py
main.py
puzzle.py
requirements.txt
.git\index
.git\config
```

Tool call: **Read** README.md

Tool call: **Read** game.py

Result

```
1      # Scenario 22 – Sliding Puzzle
2
3      A terminal sliding puzzle with configurable board size and legal blank-space moves.
4
5      ## Provided files
6
7      - `main.py` – entry point.
8      - `game.py` – interaction, timer, and move tracking.
9      - `puzzle.py` – puzzle representation and movement rules.
10     - `requirements.txt` – dependency declaration.
11
12     ## Setup
13
14     ```bash
15     python main.py
16     ```
17
18     ## Before changing the code
19
20     Start several puzzles and inspect the initial boards. Think about whether arbitrary
21     permutations can always be reached from the solved state using legal moves.
22
23     ## Task 1 – Guarantee solvable starting states
24
25     Change puzzle initialisation so every generated board is reachable from the solved
26     arrangement. Prefer generating the solved board and applying legal blank moves to
27     scramble it.
28
29     **Done when:** repeated fresh puzzles are solvable in principle and the puzzle never
30     starts in an unreachable parity state.
31
32     ## Task 2 – Complete puzzle lifecycle
33
34     Detect the solved arrangement and end the game cleanly. Prevent commands from changing
35     state after completion.
36
37     ## Task 3 – Size, timer, and move modes
38
39     Add 3x3, 4x4, and 5x5 modes. Track moves and elapsed time consistently across all sizes.
40     Do not reset session values accidentally when only the board is recreated.
41
42     ## Task 4 – Valid-action feedback
43
44     Only report a successful slide when a tile actually moved into the blank. Invalid moves
45     must not increment the move count or generate success feedback.
46
47     ## Required testing
48
49     Generate many fresh boards, test all sizes, solve a small board, attempt impossible
50     moves, verify move counts, verify timer behaviour, test invalid commands, and quit.
51
52
53     ## LLM usage
54
55     You may use an LLM during the lab. The goal is to use it as a coding assistant while
56     retaining responsibility for understanding and testing the result.
57
58     - Inspect the existing code before asking for changes.
59     - Ask for explanations when you do not understand a proposed change.
60     - Test generated code against the stated behaviour and edge cases.
61     - Keep your complete LLM chat history for submission.
62     - Do not replace the whole project with an unrelated implementation.
63     - Keep all state in memory; do not add CSV, JSON, SQLite, or other persistence.
64
65     ## Submission checklist
66
67     - [ ] Task 1 completed and the original defect was reproduced and fixed.
68     - [ ] T
... [truncated 810 more characters]
```

Tool call: **Read** main.py

Result

```
1 import time
2 from puzzle import Puzzle
3
4
5 class SlidingPuzzle:
6     def __init__(self):
7         self.size = 4
8         self.puzzle = Puzzle(self.size)
9         self.moves = 0
10        self.started = time.monotonic()
11
12    def display(self):
13        print()
14        for row in self.puzzle.board:
15            print(" ".join(f"{x or '':>2}" for x in row))
16        print("Moves:", self.moves, " Time:", int(time.monotonic() - self.started), "s")
17
18    def run(self):
19        print("Sliding Puzzle – W/A/S/D moves the tile into the blank. Q quits.")
20        while True:
21            self.display()
22            if self.puzzle.solved():
23                print("Solved!")
24                return
25            key = input("> ").strip().lower()
26            if key == "q":
27                return
28            if key not in "wasd":
29                print("Use W/A/S/D.")
30                continue
31            if self.puzzle.move(key):
32                self.moves += 1
33            else:
34                print("That move is not possible.")
35
```

Result

```
1 from game import SlidingPuzzle
2
3 if __name__ == "__main__":
4     SlidingPuzzle().run()
5
```

Tool call: **Read** puzzle.py

Result

```
1 import random
2
3
4 class Puzzle:
5     def __init__(self, size=4):
6         self.size = size
7         self.board = self.make_board()
8
9     def make_board(self):
10        tiles = list(range(1, self.size * self.size)) + [0]
11        random.shuffle(tiles) # intentional solvability defect
12        return [tiles[r * self.size:(r + 1) * self.size] for r in range(self.size)]
13
14    def blank_pos(self):
15        for r in range(self.size):
16            for c in range(self.size):
17                if self.board[r][c] == 0:
```

```

18             return r, c
19
20     def move(self, direction):
21         r, c = self.blank_pos()
22         dr, dc = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}[direction]
23         nr, nc = r + dr, c + dc
24         if not (0 <= nr < self.size and 0 <= nc < self.size):
25             return False
26         self.board[r][c], self.board[nr][nc] = self.board[nr][nc], self.board[r][c]
27         return True
28
29     def solved(self):
30         return sum(self.board, []) == list(range(1, self.size * self.size)) + [0]
31

```

Tool call: **Read** requirements.txt

Result

```

1      # Standard-library-only scenario.
2      # No third-party packages are required.
3      # Recommended Python version: 3.9+
4

```

I read all five files: README.md, main.py, game.py, puzzle.py and requirements.txt. I haven't changed anything.

The task

This is Lab Week 4, Scenario 22: a terminal sliding puzzle (the 15-puzzle family) written in Python with only the standard library. The starter code works but has a deliberate solvability defect and several missing lifecycle features. The README asks you to find and fix them without rewriting the project.

The code

File	Role
main.py	Entry point. It runs <code>SlidingPuzzle().run()</code> .
game.py	<code>SlidingPuzzle</code> handles the input loop, display, move counter and timer. The size is hard-coded to 4.
puzzle.py	<code>Puzzle</code> holds the board, <code>make_board</code> , <code>blank_pos</code> , <code>move</code> (WASD moves the blank) and <code>solved</code> .
requirements.txt	Comments only. No dependencies are needed (Python 3.9+).

The four tasks

Task 1: guarantee solvable starts. - `make_board` calls `random.shuffle`, which the code marks as an "intentional solvability defect". About half of the shuffled boards are unreachable from the solved state. - The fix is to start from the solved board and apply random legal blank moves. That keeps the parity valid by construction. - It is done when every fresh board is solvable and none starts in an unreachable parity state. - The README says to reproduce the defect first. You can do that by checking inversion parity, or by trying a few boards.

Task 2: complete lifecycle. - Detect the solved board and end the game cleanly. - After completion, no command may change state. - Today `run()` checks `solved()` at the top of the loop and returns, but the timer

keeps counting. - A freshly generated board could also start already solved.

Task 3: size, timer and move modes. - Add 3x3, 4x4 and 5x5 modes. - Moves and elapsed time must work the same across sizes. - Recreating the board must not reset the session's moves or timer by accident. - Right now `self.puzzle = Puzzle(...)` and the counters live together in `__init__`, so a naive size switch would reset everything.

Task 4: valid-action feedback. - Report success only when a tile actually moved, and don't touch the move count or print success feedback for invalid moves. - `move()` already returns `False` at the edges, so the count is mostly right. - Nothing prints on success yet, and invalid input isn't handled properly (see below).

Issues I noticed beyond the README

1. **Input crash.** `if key not in "wasd"` is a substring test, so an empty line, `"wa"`, `"as"` or `"wasd"` passes the check. They then reach `Puzzle.move`, and the `[direction]` dict lookup raises `KeyError`. This is the "test invalid commands" case.
2. **Ambiguous direction semantics.** The banner says WASD "moves the tile into the blank". The code moves the blank in that direction, so pressing `w` makes the tile above it slide down. The labelling and behaviour should be made consistent and documented.
3. **Timer.** It starts in `__init__`, shows only when the board is redrawn, and never stops on completion.
4. **README inconsistencies.** The folder tree says `scenario-10-sliding-puzzle/` while the title says Scenario 22. This is cosmetic.

Constraints

- Keep all state in memory, with no CSV, JSON or SQLite.
- Add no external dependencies.
- Keep the code modular and keep the existing structure.
- Keep the full LLM chat history.

Required testing

- Generate many fresh boards and check solvability.
- Test all three sizes.
- Solve a small board.
- Try impossible moves.
- Verify the move counts.
- Verify the timer.
- Test invalid commands, and quit.

Deliverables

Submission is three things: - A ~10-second gameplay video before the changes, showing the bug. - A ~10-second video after the changes, showing the fix and the new features. - A link to the complete chat history.

I'll wait for your go-ahead before touching the code. If you want a baseline recording first, I can write a short script that reproduces the unsolvable-board defect so you can capture the "before" video.

@"..\\software-engineering-lab-material_sec_h\\lab4\\Lab_4_VibeCoding_Student_handout.pdf"
these are the instructions given to us in general. just go through them once. do not get working yet.

ASSISTANT - CLAUDE SONNET 5.5 (CLAUDE CODE)

The PDF content didn't come through in that message, so I'll read it myself.

Tool call: **Read** ..\\software-engineering-lab-material_sec_h\\lab4\\Lab_4_VibeCoding_Student_handout.pdf (pages 1-10)

Result

pdftoppm is not installed. Install poppler-utils (e.g. `brew install poppler` or `apt-get install poppler-utils`) to enable PDF page rendering.

Page rendering isn't available, so I'll try extracting the text directly.

Tool call: **Bash** Check available PDF text extraction tools

```
cd "..\\software-engineering-lab-material_sec_h\\lab4" && (python -c "import pypdf" 2>&1; python -c "import fitz" 2>&1; which pdftotext)
```

Result

```
/ucrt64/bin/pdftotext  
Shell cwd was reset to .
```

Tool call: **Bash** Extract text from the handout PDF

```
pdftotext -layout "..\\software-engineering-lab-material_sec_h\\lab4\\Lab_4_VibeCoding_Student_handout.pdf" -
```

Result

Lab 4: VibeCoding

Objective

Use Vibe Coding tools to fix the broken code and add features to the same. Learn how to prompt to get the job done in under three to four attempts. Clone a repo assigned to you, make changes and push to your repo in your personal account.

THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!!

Duration

60 minutes

Software Requirements

Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc)

Instructions:

1. Go to the repo assigned to you
2. Go through the ReadMe file to understand the deliverables
3. Clone or Fork the repo
4. Run the Python based code (A game/ A Puzzle)
5. Record 10 seconds of video before making any changes.
6. Write prompts and fix the broken code
7. Write prompts to add all the features listed in the ReadMe file of the repo one by one.
8. Record another 10 seconds of video again after making all the changes.
9. Push the code to your repo.
10. Have Separate Commits for each task
11. Do not raise PR to the main repo (SETAPESU26)

Deliverables:

Upload the following to your Lab repo under Lab-4 folder,

- a. Videos (before and after)
- b. Updated Code
- c. Chat history exported as a doc/pdf

I read the handout. It's the general Lab 4 "VibeCoding" brief, and it sits on top of the scenario README.

Objective

Use an AI coding tool (ChatGPT, Gemini, Claude, etc.) to fix the broken code and add the listed features. The aim is to learn to prompt well enough to get each job done in about three to four attempts. This is an **individual** assignment and runs **60 minutes**.

Steps

1. Go to your assigned repo and read the README to understand the deliverables.
2. Clone or fork it.
3. Run the game.
4. **Record 10 seconds of video before any changes.**

5. Prompt the tool to fix the broken code.
6. Prompt it to add each README feature **one by one**.
7. **Record another 10 seconds of video after all changes.**
8. Push to your own repo in your personal account.
9. Make **a separate commit for each task**.
10. **Do not raise a PR** to the main repo (SETAPESU26).

Deliverables

Upload to your lab repo under a **Lab-4** folder: - The before and after videos. - The updated code. - The chat history, exported as a doc or PDF.

How it fits with the scenario README

- **Submission:** the handout lists the code and chat history as repo uploads. The scenario README says submission is two videos plus a chat-history link. The handout says the chat history goes in the repo as a doc or PDF, so I'd follow it and include the link too if your instructor asks.
- **Commits:** I'd make one commit each for Task 1 (solvable starts), Task 2 (lifecycle), Task 3 (size, timer and moves) and Task 4 (valid-action feedback).
- **Prompting:** the three-to-four-attempts goal means focused prompts per task, not one big rewrite. That matches the README's rule not to replace the whole project.
- **Repo location:** this local folder has a git remote `origin`, which may be the assigned repo. You'll need your own personal repo to push to, and I haven't checked where `origin` points.
- **Videos:** I can't record them. The "before" one needs to be taken now, while the code is still broken.

I haven't changed any code or run anything on the project. Tell me when to start. Before that, I can show you what `origin` points to, or give you a short script that reliably shows the unsolvable-board bug for the "before" recording.

SLASH COMMAND

```
/model claude-opus-5-5
```

Set model to `claude-opus-5-5`

USER - PROMPT 3

so what is the final plan of action before getting started, to finish the given tasks in the exact order?

ASSISTANT - CLAUDE OPUS 5.5 (CLAUDE CODE)

Tool call: **Bash** Show git remotes and recent commits

```
git remote -v; git log --oneline -3
```

Result

```
origin https://github.com/pranav-prabhakar-dev/22_sliding_puzzle.git (fetch)
origin https://github.com/pranav-prabhakar-dev/22_sliding_puzzle.git (push)
```

Here's the plan, in the order the handout and README require. Your `origin` points to `pranav-prabhakar-dev/22_sliding_puzzle`, which is your personal account, so pushing there is fine.

Step 0: Before any changes (you do this)

1. Run `python main.py` and **record the 10-second "before" video**. The README hints that some starting boards can't be solved, but that's hard to see on screen. Two bugs you can actually show: - Pressing Enter on an empty line crashes the game with a `KeyError`. - The game only offers a 4x4 board.
2. I'll write a throwaway script in my scratchpad, not the repo. It generates about 1,000 boards and counts the unsolvable ones (expect about 50%). That reproduces the defect, which the submission checklist requires.

Task 1: Solvable starting boards (commit 1)

- **Change:** `Puzzle.make_board()` builds the solved board, then makes many random legal blank moves to scramble it. It won't immediately undo the previous move, and it re-scrambles if the result happens to be already solved. `random.shuffle` goes away.
- **Check:** run the parity script again, and expect 0 unsolvable boards for sizes 3, 4 and 5.

Task 2: Game lifecycle (commit 2)

- **Change:** `SlidingPuzzle` gets a `finished` flag. When the board is solved, it freezes the timer, prints the final moves and time, and ends cleanly. Once finished, any move command is rejected and nothing changes.
- **Check:** solve a 3x3 board by script (feed it the reverse of the scramble), then confirm that later commands don't change the board, the move count or the time.

Task 3: Sizes, timer and move tracking (commit 3)

- **Change:**
- At startup you pick 3x3, 4x4 or 5x5.
- A command such as `n` starts a new board, optionally at a different size.
- The puzzle's state is kept separate from the session's state.
- One method, `elapsed()`, works out the time, so the time stays consistent and freezes when the game is solved.
- **Check:** every size shows correctly (5x5 still lines up), the timer keeps increasing and freezes on solve, and creating a new board resets only what it should.

Task 4: Valid-move feedback (commit 4)

- **Change:**
- A command counts only if it is exactly one of `w`, `a`, `s` or `d`. This fixes the crash on an empty line, `wa` or `wasd`.
- The game prints a success message such as "Moved tile 7" only when a tile actually moved.
- Edge moves, invalid commands and commands after the game is finished leave the move count unchanged and print an error.

- **Check:** try every edge move and junk inputs like an empty line, `x`, `wa` and `q`, and confirm the move counts.

Final steps

1. Run the full test list from the README: many boards, all sizes, solving a small board, impossible moves, move counts, the timer, invalid commands and quitting.
2. **You record the 10-second "after" video.**
3. Push the four commits to `origin/main`. No PR to SETAPESU26.
4. **You export the chat** as a doc or PDF, then upload the videos, the code and the chat to the `Lab-4` folder of your lab repo. That may be a different repo from this one.

Two decisions I need from you before starting

1. **What the keys move.** The game's title line says the keys move a tile into the blank, but `w` actually moves the blank up, so the tile above it slides down. I recommend making the code match the title line, so `w` slides the tile below the blank up. The other option is to keep the current behaviour and fix the wording instead.
2. **What a new board resets.** I recommend that a new board resets that game's moves and timer, while totals for the whole session (games played, games solved, total moves) carry over. If you read "Do not reset session values" more strictly, the alternative is that moves and time carry over across new boards too.

Tell me your choices, or say "go with recommendations", and I'll start with Task 1 once your "before" video is recorded.

USER - PROMPT 4

@"BEFORE.mp4"

This is the before video. I did task 0

ASSISTANT - CLAUDE OPUS 5.5 (CLAUDE CODE)

Thanks. I can't play video files, so I haven't watched it, but Step 0 is done. Neither decision affects Task 1, so I'll start with the defect reproduction and Task 1. I'll ask for your two choices before Task 3.

Tool call: **Write** `scratchpad\parity_check.py`

```
"""Count how many freshly generated boards are unsolvable, per size."""
import sys

sys.path.insert(0, sys.argv[1])
from puzzle import Puzzle

def is_solvable(board):
    size = len(board)
    flat = [x for row in board for x in row]
    tiles = [x for x in flat if x]
    inversions = sum(
        1 for i in range(len(tiles)) for j in range(i + 1, len(tiles)) if tiles[i] > tiles[j]
    )
    if size % 2 == 1:
        return inversions % 2 == 0
    blank_row_from_bottom = size - flat.index(0) // size
    return (inversions + blank_row_from_bottom) % 2 == 1
```

```

N = 1000
for size in (3, 4, 5):
    bad = solved = 0
    for _ in range(N):
        p = Puzzle(size)
        bad += not is_solvable(p.board)
        solved += p.solved()
    print(f"{size}x{size}: {bad}/{N} unsolvable, {solved}/{N} start already solved")

```

Result

File created successfully at: scratchpad\parity_check.py (file state is current in your context – no need to Read it back)

Tool call: **Bash** Reproduce the unsolvable-board defect on original code

```
python "scratchpad/parity_check.py" .
```

Result

```

3x3: 516/1000 unsolvable, 0/1000 start already solved
4x4: 504/1000 unsolvable, 0/1000 start already solved
5x5: 502/1000 unsolvable, 0/1000 start already solved

```

The defect is reproduced: about 50% of shuffled boards can't be solved at every size. Now the fix:

Tool call: **Edit** puzzle.py

Replace:

```

import random

class Puzzle:
    def __init__(self, size=4):
        self.size = size
        self.board = self.make_board()

    def make_board(self):
        tiles = list(range(1, self.size * self.size)) + [0]
        random.shuffle(tiles) # intentional solvability defect
        return [tiles[r * self.size:(r + 1) * self.size] for r in range(self.size)]

```

With:

```

import random

DIRECTIONS = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}
OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}

class Puzzle:
    def __init__(self, size=4):
        self.size = size
        self.board = self.make_board()

    def make_board(self):
        # Start solved and scramble with legal blank moves, so every board is reachable.
        tiles = list(range(1, self.size * self.size)) + [0]
        self.board = [tiles[r * self.size:(r + 1) * self.size] for r in range(self.size)]
        while self.solved():
            self.scramble(self.size * self.size * 20)
        return self.board

    def scramble(self, steps):
        last = None

```

```

    for _ in range(steps):
        options = [d for d in DIRECTIONS if d != OPPOSITE.get(last) and self.can_move(d)]
        last = random.choice(options)
        self.move(last)

    def can_move(self, direction):
        r, c = self.blank_pos()
        dr, dc = DIRECTIONS[direction]
        return 0 <= r + dr < self.size and 0 <= c + dc < self.size

```

Result

The file puzzle.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: **Edit** puzzle.py

Replace:

```

r, c = self.blank_pos()
dr, dc = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}[direction]
nr, nc = r + dr, c + dc
if not (0 <= nr < self.size and 0 <= nc < self.size):
    return False
self.board

```

With:

```

if not self.can_move(direction):
    return False
r, c = self.blank_pos()
dr, dc = DIRECTIONS[direction]
nr, nc = r + dr, c + dc
self.board

```

Result

The file puzzle.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: **Bash** Re-run parity check, smoke-test the game, show changes

```
python "scratchpad/parity_check.py" . && printf 'w\na\ns\nd\nq\n' | python main.py | tail -8 && git status --short
```

Result

```

3x3: 0/1000 unsolvable, 0/1000 start already solved
4x4: 0/1000 unsolvable, 0/1000 start already solved
5x5: 0/1000 unsolvable, 0/1000 start already solved

Moves: 2   Time: 0 s

>
  2  7  1  4
15 11      3
  6  9 13  5
14 10  8 12

Moves: 3   Time: 0 s

> M puzzle.py

```

```
?? BEFORE.mp4
?? __pycache__/
```

I verified the fix: 0 out of 1,000 boards are unsolvable at every size, and the game still runs. I'll commit only `puzzle.py`. `BEFORE.mp4` belongs in your Lab-4 folder, not in this commit.

Tool call: **Bash** Commit Task 1 changes to `puzzle.py`

```
git add puzzle.py && git commit -q -F - <<'EOF'
Task 1: Guarantee solvable starting boards
```

Replace `random.shuffle` (which produced unreachable parity states ~50% of the time) with a scramble of legal blank moves applied to the solved board. Re-scramble if the result happens to be already solved.

```
Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>
EOF
git log --oneline -1
```

Result

```
045a459 Task 1: Guarantee solvable starting boards
```

Task 1 is done and committed locally as `045a459`. It's not pushed yet.

Reproducing the bug: I generated 1,000 boards per size with the original code. About half couldn't be solved: 516 of 1,000 at 3x3, 504 at 4x4 and 502 at 5x5.

Fix: in `puzzle.py`, `make_board()` now starts from the solved board and makes `size2 × 20` random legal moves of the blank. It never immediately undoes the previous move, and it scrambles again if the board ends up already solved. I also added a `can_move()` helper and moved the direction table into a constant, `DIRECTIONS`, that the `scramble` and `move()` share.

Check: after the fix, 0 of 1,000 boards are unsolvable at every size, and none start already solved. A quick run of the game with a few moves and `q` worked normally.

`BEFORE.mp4` and `__pycache__` are untracked in this folder, and I left them out of the commit.

I've stopped here so each task gets its own prompt, as the handout says. Before Tasks 3 and 4, I still need your two choices: 1. **What the keys move:** should `w` slide the tile below the blank up, as the title line says (my recommendation)? Or keep the current behaviour, where `w` moves the blank up, and fix the wording instead? 2. **What a new board resets:** should it reset that game's moves and timer while totals for the whole session carry over (my recommendation)? Or should moves and time carry over across new boards too?

Say "continue with Task 2" whenever you're ready, and include your choices.

USER - PROMPT 5

okay task 2 now

ASSISTANT - CLAUDE OPUS 5.5 (CLAUDE CODE)

Task 2 only touches the game loop in `game.py`. I'll add a finished state, freeze the timer when the puzzle is solved, and block moves once it's done. The input-validation crash stays for Task 4, so it lands in its own commit.

Tool call: **Edit** game.py

Replace:

```
self.moves = 0
self.started = time.monotonic()

def display(self):
    print()
    for row in self.puzzle.board:
        print(" ".join(f"{x or ' ':>2}" for x in row))
    print("Moves:", self.moves, " Time:", int(time.monotonic() - self.started), "s")

def run(self):
    print("Sliding Puzzle – W/A/S/D moves the tile into the blank. Q quits.")
    while True:
        self.display()
        if self.puzzle.solved():
            print("Solved!")
            return
        key = input("> ").strip().lower()
        if key == "q":
            return
        if key not in "wasd":
            print("Use W/A/S/D.")
            continue
        if self.puzzle.move(key):
            self.moves += 1
        else:
            print("That move is not possible.")
```

With:

```
self.moves = 0
self.started = time.monotonic()
self.finished_at = None

@property
def finished(self):
    return self.finished_at is not None

def elapsed(self):
    # The clock stops once the puzzle is solved.
    end = self.finished_at if self.finished else time.monotonic()
    return int(end - self.started)

def display(self):
    print()
    for row in self.puzzle.board:
        print(" ".join(f"{x or ' ':>2}" for x in row))
    print("Moves:", self.moves, " Time:", self.elapsed(), "s")

def slide(self, key):
    if self.finished:
        print("The puzzle is already solved; no more moves allowed.")
        return False
    if not self.puzzle.move(key):
        print("That move is not possible.")
        return False
    self.moves += 1
    if self.puzzle.solved():
        self.finished_at = time.monotonic()
    return True

def run(self):
    print("Sliding Puzzle – W/A/S/D moves the tile into the blank. Q quits.")
    while True:
        self.display()
        if self.finished:
            print(f"Solved in {self.moves} moves and {self.elapsed()} s!")
            return
        key = input("> ").strip().lower()
        if key == "q":
            print("Quit without solving.")
            return
        if key not in "wasd":
```

```
print("Use W/A/S/D.")
continue
self.slide(key)
```

Result

The file game.py has been updated successfully. (file state is current in your context – no need to Read it back)

Now a test script that solves a board end to end and then tries to change state after completion:

Tool call: **Write** scratchpad/task2_check.py

```
"""Task 2 checks: solve detection, clean end, frozen state after completion."""
import builtins
import copy
import random
import sys
import time

sys.path.insert(0, sys.argv[1])
import game
from game import SlidingPuzzle
from puzzle import OPPOSITE, Puzzle

# 1. Solve a small board end to end through run(): scramble 3x3 with a recorded
# walk, then feed the reversed walk as user input.
random.seed(7)
g = SlidingPuzzle()
g.puzzle = Puzzle(3)
g.puzzle.board = [[1, 2, 3], [4, 5, 6], [7, 8, 0]]
walk, last = [], None
while len(walk) < 12 or g.puzzle.solved():
    opts = [d for d in "wasd" if d != OPPOSITE.get(last) and g.puzzle.can_move(d)]
    last = random.choice(opts)
    g.puzzle.move(last)
    walk.append(last)
solution = [OPPOSITE[d] for d in reversed(walk)]
inputs = iter(solution + ["w", "a", "s", "d"]) # extra keys must never be read
builtins.input = lambda prompt="": next(inputs)
g.run()
leftover = list(inputs)
assert g.finished and g.puzzle.solved(), "board should be solved"
assert g.moves == len(solution), (g.moves, len(solution))
assert leftover == ["w", "a", "s", "d"], "run() kept reading input after solve"
print(f"PASS solved 3x3 in {g.moves} moves; run() ended cleanly")

# 2. Commands after completion must not change board, moves, or time.
board_before, moves_before = copy.deepcopy(g.puzzle.board), g.moves
real_monotonic = time.monotonic
game.time.monotonic = lambda: real_monotonic() + 100 # pretend 100 s passed
t_before = g.elapsed()
for key in "wasd":
    assert g.slide(key) is False
assert g.puzzle.board == board_before and g.moves == moves_before
assert g.elapsed() == t_before, "timer kept running after solve"
game.time.monotonic = real_monotonic
print("PASS post-solve commands rejected; board, moves, and timer frozen")

# 3. Timer still runs while unsolved.
g2 = SlidingPuzzle()
game.time.monotonic = lambda: real_monotonic() + 5
assert g2.elapsed() >= 5 and not g2.finished
game.time.monotonic = real_monotonic
print("PASS timer runs while unsolved")
```

Result

File created successfully at: scratchpad/task2_check.py (file state is current in your context – no need to Read it back)

Tool call: **Bash** Run Task 2 checks and a quit smoke test

```
python "scratchpad/task2_check.py" . | grep -E "PASS|Solved|already" ; printf 'w\nq\n' | python main.py | tail -2
```

Result

```
Solved in 12 moves and 0 s!
PASS solved 3x3 in 12 moves; run() ended cleanly
The puzzle is already solved; no more moves allowed.
The puzzle is already solved; no more moves allowed.
The puzzle is already solved; no more moves allowed.
The puzzle is already solved; no more moves allowed.
PASS post-solve commands rejected; board, moves, and timer frozen
PASS timer runs while unsolved
Moves: 1 Time: 0 s

> Quit without solving.
```

All checks pass. Committing Task 2:

Tool call: **Bash** Commit Task 2 changes to game.py

```
git add game.py && git commit -q -F - <<'EOF'
Task 2: Complete puzzle lifecycle
```

Track a finished state: when a move solves the board, stop the timer, report the final moves and time, and end the game cleanly. Any slide attempted after completion is rejected without changing the board, the move count, or the elapsed time.

```
Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>
EOF
git log --oneline -2
```

Result

```
ee093a2 Task 2: Complete puzzle lifecycle
045a459 Task 1: Guarantee solvable starting boards
```

Task 2 is done and committed locally as ee093a2. Nothing is pushed yet.

What changed in [game.py](#):

- **Solve detection:** when a valid slide solves the board, the game stops the timer at that moment. It shows the final board, prints "Solved in N moves and T s!" and exits the loop.
- **Locked after solving:** moves now go through a new `slide()` method. Once the puzzle is solved, it refuses every move with a message, and the board, move count and time stay unchanged.
- **One timer:** a single `elapsed()` method now gives the time everywhere. That sets up Task 3, which needs the time to behave the same at every size.
- **Quitting:** `q` now prints "Quit without solving." instead of exiting silently.

Tests (a script in my scratchpad, not the repo):

- I scrambled a 3x3 board with 12 recorded moves and fed the reverse as keyboard input to the real game loop. It detected the solve after exactly 12 moves and ended without reading the 4 extra keys queued after the solution.
- After solving, I sent all 4 move keys directly to `slide()`. Each was rejected, and I simulated 100 seconds passing: the board, the move count (12) and the time stayed the same.
- On an unsolved board the timer keeps running, and quitting still works.

The game still offers only 4x4, so you can't reach the solved screen yourself yet; this lifecycle is checked only by the script. The crash on an empty or multi-letter command is still there on purpose, because Task 4 fixes it.

Next is Task 3: board sizes, timer and move tracking. It needs the second decision from earlier: should a new board reset that game's moves and timer while totals for the whole session carry over (my recommendation)? Or should moves and time carry over across boards? Task 4 then needs the first decision: should we slide the tile below the blank up, as the title line says (my recommendation), or keep the current behaviour and fix the wording?

USER - PROMPT 6

Before you continue with the next task, just to make sure, all instructions are being adhered to right? You may continue after you clarify that with me.

ASSISTANT - CLAUDE OPUS 5.5 (CLAUDE CODE)

Yes. Here's the check against both documents. Nothing breaks the rules so far; some items are still to come, and two small points are worth knowing about.

Lab handout

Instruction	Status
Read the README first	✓ Done before touching any code
Run the game and record a 10-second "before" video	✓ You recorded BEFORE.mp4 before any code changed
Fix the broken code, then add features one by one	✓ Tasks 1 and 2 were each done on their own prompt, in README order
Get each job done in about 3–4 prompts	✓ Each task so far took one prompt
A separate commit for each task	✓ 045a459 is Task 1 and ee093a2 is Task 2; each touches only its task's files
Push to your own repo	⌚ Not pushed yet. origin is pranav-prabhakar-dev/22_sliding_puzzle, your personal account
No PR to SETAPESU26	✓ No PR, and none planned
Record a 10-second "after" video	⌚ After Task 4
Upload videos, code and chat export to the Lab-4 folder	⌚ At the end (yours to do)

Scenario README

Requirement	Status
Inspect the code before changing it	✓
Task 1: original defect reproduced and fixed	✓ About 50% of boards were unsolvable before, 0 of 1,000 after, at all sizes
Task 2: detect the solve, end cleanly, block changes after	✓ Tested by script

Requirement	Status
Don't replace the project; keep it modular	✓ Same three files and the same <code>Puzzle / SlidingPuzzle</code> split; only small, targeted edits
Keep all state in memory, no files or databases	✓
No unnecessary external dependencies	✓ Only <code>random</code> and <code>time</code> from the standard library
Keep the complete chat history	✓ This session is it. Export it at the end

Points worth knowing

1. **Small overlap between Tasks 2 and 3.** Task 2 needed the timer to stop when the puzzle is solved. So I added the `elapsed()` method then, though timer consistency is part of Task 3. That keeps the Task 2 commit working on its own, and Task 3 will build on it.
2. **The test scripts are in my scratchpad, not your repo.** The README requires the tests to be done, not committed, so I've kept the repo clean. If you'd like them in the repo as proof, I can add a `tests/` folder in its own commit.
3. **BEFORE.mp4 and `__pycache__` / sit untracked in this folder.** I'm keeping them out of every commit.
4. **Where Lab-4 lives.** The handout says to upload to "your Lab repo under Lab-4 folder". If that's a separate repo from `22_sliding_puzzle`, the code will need copying there at the end. We can settle that then.
5. **The two submission lists differ.** The README asks for two videos plus a chat link. The handout asks for videos, code and the chat as a doc or PDF. Covering the handout's list satisfies both.

Before Task 3 I still need your two choices:

Tool call: **AskUserQuestion** Question(s) for the user

Task 3: when a new board is created (new game or size change), what should happen to moves and time?

- **Per-game reset (Recommended)** - The new board's own moves and timer start at 0. Totals for the whole session (games played, games solved, total moves) carry over. Creating a board can't touch the session values by accident.
- **Carry everything over** - Moves and time keep counting across all boards in the session. A size change only swaps the board.

Task 4: what should W/A/S/D move?

- **Move the tile (Recommended)** - Matches the title line: W slides the tile below the blank up into it. This flips the current key behaviour.
- **Move the blank** - Keep the current behaviour (W moves the blank up) and change the title line to say so.

Result

The user answered: "Task 3: when a new board is created (new game or size change), what should happen to moves and time?"="whatever is suitable", "Task 4: what should W/A/S/D move?"="whatever is suitable". Read the answers carefully – they may request clarification, changes, or that you not proceed – and follow what they actually say.

Tool call: **Read** game.py

Result

```
1     import time
2     from puzzle import Puzzle
3
4
5     class SlidingPuzzle:
6         def __init__(self):
7             self.size = 4
8             self.puzzle = Puzzle(self.size)
9             self.moves = 0
10            self.started = time.monotonic()
11            self.finished_at = None
12
13        @property
14        def finished(self):
15            return self.finished_at is not None
16
17        def elapsed(self):
18            # The clock stops once the puzzle is solved.
19            end = self.finished_at if self.finished else time.monotonic()
20            return int(end - self.started)
21
22        def display(self):
23            print()
24            for row in self.puzzle.board:
25                print(" ".join(f"{x or ' ':>2}" for x in row))
26            print("Moves:", self.moves, " Time:", self.elapsed(), "s")
27
28        def slide(self, key):
29            if self.finished:
30                print("The puzzle is already solved; no more moves allowed.")
31                return False
32            if not self.puzzle.move(key):
33                print("That move is not possible.")
34                return False
35            self.moves += 1
36            if self.puzzle.solved():
37                self.finished_at = time.monotonic()
38            return True
39
40        def run(self):
41            print("Sliding Puzzle – W/A/S/D moves the tile into the blank. Q quits.")
42            while True:
43                self.display()
44                if self.finished:
45                    print(f"Solved in {self.moves} moves and {self.elapsed()} s!")
46                    return
47                key = input("> ").strip().lower()
48                if key == "q":
49                    print("Quit without solving.")
50                    return
51                if key not in "wasd":
52                    print("Use W/A/S/D.")
53                    continue
54                self.slide(key)
55
```

Tool call: **Write** game.py

```
import time
from puzzle import Puzzle

SIZES = (3, 4, 5)
DEFAULT_SIZE = 4

class SlidingPuzzle:
    def __init__(self):
        # Session values: kept for the whole run, never touched by new_board().
        self.session_started = time.monotonic()
        self.boards_played = 0
        self.boards_solved = 0
```

```

        self.total_moves = 0
        self.size = DEFAULT_SIZE
        self.puzzle = None

    def new_board(self, size):
        # Per-board values only: a fresh board starts its own move count and clock.
        self.size = size
        self.puzzle = Puzzle(size)
        self.moves = 0
        self.started = time.monotonic()
        self.finished_at = None
        self.boards_played += 1

    @property
    def finished(self):
        return self.finished_at is not None

    def elapsed(self):
        # The clock stops once the puzzle is solved.
        end = self.finished_at if self.finished else time.monotonic()
        return int(end - self.started)

    def session_elapsed(self):
        return int(time.monotonic() - self.session_started)

    def ask_size(self):
        """Return a board size, or None if the player wants to quit."""
        while True:
            choice = input(f"Board size 3, 4 or 5 [Enter = {self.size}, Q quits]: ").strip().lower()
            if choice == "q":
                return None
            if choice == "":
                return self.size
            if choice.isdigit() and int(choice) in SIZES:
                return int(choice)
            print("Please enter 3, 4 or 5.")

    def display(self):
        print()
        for row in self.puzzle.board:
            print(" ".join(f"{x or ' ':>2}" for x in row))
        print(f"Size: {self.size}x{self.size} Moves: {self.moves} Time: {self.elapsed()} s")
        if self.finished:
            print(f"Solved in {self.moves} moves and {self.elapsed()} s! N for a new board, Q quits.")

    def session_summary(self):
        print(
            f"Session: {self.boards_played} board(s), {self.boards_solved} solved, "
            f"{self.total_moves} total moves, {self.session_elapsed()} s."
        )

    def slide(self, key):
        if self.finished:
            print("The puzzle is already solved; no more moves allowed.")
            return False
        if not self.puzzle.move(key):
            print("That move is not possible.")
            return False
        self.moves += 1
        self.total_moves += 1
        if self.puzzle.solved():
            self.finished_at = time.monotonic()
            self.boards_solved += 1
        return True

    def quit(self):
        if self.puzzle is not None and not self.finished:
            print("Quit without solving.")
            self.session_summary()

    def run(self):
        print("Sliding Puzzle – W/A/S/D moves the tile into the blank. N new board, Q quits.")
        size = self.ask_size()
        if size is None:
            return self.quit()

```

```

self.new_board(size)
while True:
    self.display()
    key = input("> ").strip().lower()
    if key == "q":
        return self.quit()
    if key == "n":
        size = self.ask_size()
        if size is None:
            return self.quit()
        self.new_board(size)
        continue
    if key not in "wasd":
        print("Use W/A/S/D, N or Q.")
        continue
    self.slide(key)

```

Result

The file game.py has been updated successfully. (file state is current in your context – no need to Read it back)

Now a test script for Task 3. It also re-runs the Task 2 checks, because after a solve the game now waits for N or Q instead of exiting.

Tool call: **Write** scratchpad\task3_check.py

```

"""Task 3 checks (plus Task 2 regression): sizes, timer, per-board vs session values."""
import builtins
import random
import sys
import time

sys.path.insert(0, sys.argv[1])
import game
from game import SlidingPuzzle
from puzzle import OPPOSITE

real_monotonic = time.monotonic
clock = [0.0]
game.time.monotonic = lambda: real_monotonic() + clock[0] # controllable fake clock

def feed(keys):
    it = iter(keys)
    builtins.input = lambda prompt="": next(it)
    return it

def scramble_recorded(g, steps, seed):
    """Reset g's board to solved, apply a recorded walk, return the solving keys."""
    rng = random.Random(seed)
    n = g.size
    tiles = list(range(1, n * n)) + [0]
    g.puzzle.board = [tiles[r * n:(r + 1) * n] for r in range(n)]
    walk, last = [], None
    while len(walk) < steps or g.puzzle.solved():
        opts = [d for d in "wasd" if d != OPPOSITE.get(last) and g.puzzle.can_move(d)]
        last = rng.choice(opts)
        g.puzzle.move(last)
        walk.append(last)
    return [OPPOSITE[d] for d in reversed(walk)]

# 1. Every size produces a correctly shaped board containing each tile once.
for size in game.SIZES:
    g = SlidingPuzzle()
    g.new_board(size)
    flat = sorted(sum(g.puzzle.board, []))
    assert len(g.puzzle.board) == size and all(len(r) == size for r in g.puzzle.board)
    assert flat == list(range(size * size)) and not g.puzzle.solved()

```

```

print("PASS 3x3, 4x4, 5x5 boards well formed")

# 2. Size prompt: invalid input re-prompts, Enter keeps current, Q quits.
g = SlidingPuzzle()
feed(["", ]); assert g.ask_size() == 4
feed(["7", "abc", "-3", "3"]); assert g.ask_size() == 3
feed(["Q"]); assert g.ask_size() is None
print("PASS size prompt validates input")

# 3. Timer runs per board, freezes on solve, same logic for every size.
for size in game.SIZES:
    g = SlidingPuzzle()
    g.new_board(size)
    solution = scramble_recorded(g, 10, seed=size)
    clock[0] += 7
    assert g.elapsed() == 7, g.elapsed()
    for k in solution:
        assert g.slide(k)
    assert g.finished and g.moves == len(solution)
    clock[0] += 50
    assert g.elapsed() == 7, "timer must freeze on solve"
print("PASS timer runs and freezes identically for all sizes")

# 4. New board resets per-board values but not session values.
g = SlidingPuzzle()
g.new_board(3)
sol = scramble_recorded(g, 8, seed=1)
for k in sol:
    g.slide(k)
solved_moves = g.moves
session_start = g.session_started
clock[0] += 30
g.new_board(5)
assert g.moves == 0 and g.elapsed() == 0 and not g.finished and g.size == 5
assert g.boards_played == 2 and g.boards_solved == 1 and g.total_moves == solved_moves
assert g.session_started == session_start and g.session_elapsed() >= 30
print("PASS new board resets per-board values only; session values kept")

# 5. Full run(): pick 3x3, solve it, moves rejected after solve, N -> 4x4, quit.
g = SlidingPuzzle()
orig_new_board = g.new_board
solutions = []

def new_board_with_known_scramble(size):
    orig_new_board(size)
    if size == 3:
        solutions.extend(scramble_recorded(g, 12, seed=42))

g.new_board = new_board_with_known_scramble
# Pre-compute solution by creating the board via the first input "3".
inputs = ["3"]
builtins.input = lambda prompt="": inputs.pop(0)
# After the size is chosen we know the solution; append the rest lazily.
real_input = builtins.input

def scripted(prompt=""):
    if not inputs:
        inputs.extend(solutions + ["w", "d", "n", "4", "q"])
    return inputs.pop(0)

builtins.input = scripted
g.run()
assert g.boards_played == 2 and g.boards_solved == 1 and g.size == 4 and g.moves == 0
assert g.total_moves == len(solutions)
print(f"PASS full run: solved 3x3 in {len(solutions)} moves, post-solve moves rejected, "
      f"switched to 4x4, quit")

```

Result

File created successfully at: scratchpad/task3_check.py (file state is current in your context – no need to Read it back)

Tool call: **Bash** Run Task 3 checks and re-run the parity check

```
python "scratchpad/task3_check.py" . > /tmp_t3.txt 2>&1; grep -E "PASS|Error|assert|Traceback|already solved|Solved in|Session" /tmp_t3.txt; python "scratchpad/parity_check.py" .
```

Result

```
/usr/bin/bash: line 1: /tmp_t3.txt: Permission denied
grep: /tmp_t3.txt: No such file or directory
3x3: 0/1000 unsolvable, 0/1000 start already solved

4x4: 0/1000 unsolvable, 0/1000 start already solved

5x5: 0/1000 unsolvable, 0/1000 start already solved
```

I wrote the output to a path I'm not allowed to write to. Re-running with the output piped straight through:

Tool call: **Bash** Run Task 3 checks and filter key output

```
python "scratchpad/task3_check.py" . 2>&1 | grep -E "PASS|Error|Traceback|line [0-9]|already solved|Solved in|Session|Size: 4x4"
```

Result

```
PASS 3x3, 4x4, 5x5 boards well formed
PASS size prompt validates input
PASS timer runs and freezes identically for all sizes
PASS new board resets per-board values only; session values kept
Solved in 12 moves and 0 s! N for a new board, Q quits.
The puzzle is already solved; no more moves allowed.
Solved in 12 moves and 0 s! N for a new board, Q quits.
The puzzle is already solved; no more moves allowed.
Solved in 12 moves and 0 s! N for a new board, Q quits.
Size: 4x4 Moves: 0 Time: 0 s
Session: 2 board(s), 1 solved, 12 total moves, 0 s.
PASS full run: solved 3x3 in 12 moves, post-solve moves rejected, switched to 4x4, quit
```

Everything passes. Now a real run through `main.py`: a 5x5 board, then a switch to 3x3, so I can check the layout.

Tool call: **Bash** Play the game through stdin: 5x5, new 3x3 board, quit

```
printf '9\n5\nw\nn\n3\nq\n' | python main.py
```

Result

```
Sliding Puzzle 🎲 W/A/S/D moves the tile into the blank. N new board, Q quits.

Board size 3, 4 or 5 [Enter = 4, Q quits]: Please enter 3, 4 or 5.

Board size 3, 4 or 5 [Enter = 4, Q quits]:

 1  4 21  7 23
15 20 16 11 24
 3 22  2 10 18
 6  8 19    17
12 13 14  5  9
```

```

Size: 5x5 Moves: 0 Time: 0 s

>

 1  4 21  7 23
15 20 16 11 24

 3 22  2   18
 6  8 19 10 17
12 13 14  5  9

Size: 5x5 Moves: 1 Time: 0 s
> Board size 3, 4 or 5 [Enter = 5, Q quits]:

 7  6  4
 1  8  5
 3  2

Size: 3x3 Moves: 0 Time: 0 s
> Quit without solving.

Session: 2 board(s), 0 solved, 1 total moves, 0 s.

```

The 5x5 grid lines up, an invalid size (9) is re-asked, and N switches boards with the move counter starting again from 0. Committing Task 3:

Tool call: **Bash** Commit Task 3 and show status

```

git add game.py && git commit -q -F - <<'EOF'
Task 3: Add 3x3/4x4/5x5 modes with consistent moves and timer

```

Ask for the board size at start and allow N to start a new board of any supported size. Per-board values (moves, timer, finished state) live in `new_board()`; session values (boards played/solved, total moves, session time) are set once in `__init__` so recreating a board cannot reset them. After a solve the board stays locked while N and Q remain available.

```

Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>
EOF
git log --oneline -3; git status --short

```

Result

```

warning: in the working copy of 'game.py', LF will be replaced by CRLF the next time Git touches it
f06e449 Task 3: Add 3x3/4x4/5x5 modes with consistent moves and timer
ee093a2 Task 2: Complete puzzle lifecycle
045a459 Task 1: Guarantee solvable starting boards
?? BEFORE.mp4
?? __pycache__/

```

Task 3 is done and committed locally as `f06e449`. Nothing is pushed yet.

What changed in `game.py`:

- **Board sizes:** the game asks for a size (3, 4 or 5) at the start. Enter keeps the current size, Q quits, and anything else is asked again.
- **New board:** during a game, N starts a new board at any of those sizes.
- **Reset only per-board values:** each board's own moves, timer and solved state are set up in `new_board()`. Session totals (boards played and solved, total moves, session time) are set once at startup. Creating a new board can't reset them by accident.
- **Timer:** the same `elapsed()` method gives the time for

every size. It starts when a board is created and stops on the solve. - **After a solve:** the board stays locked and move keys are rejected, but you can still press N or Q. Pressing Q prints a session summary.

One change to Task 2's behaviour: after a solve, the game used to exit straight away. Now it waits for N or Q so you can play again. The solved board still can't be changed, and you can now see that on screen, which helps the "after" video.

Tests: - Boards come out the right shape at all three sizes, and the solvability check still finds 0 of 1,000 unsolvable. - The size prompt rejects 7, abc and -3. - Using a simulated clock, the timer stopped at the solve time at every size even after another 50 simulated seconds. - A new board reset the moves and timer but kept the session totals. - A full run through the game loop solved a 3x3 board in 12 moves. It then rejected moves on the solved board, switched to 4x4 and quit. - A piped run of `main.py` showed the 5x5 grid lines up.

Still to fix in Task 4: - An empty line or a multi-letter command such as `wa` still crashes the game. - A successful slide prints no message yet. - The keys still move the blank instead of the tile.

Say "Task 4" when you're ready.

USER - PROMPT 7

Task 4

ASSISTANT - CLAUDE OPUS 5.5 (CLAUDE CODE)

Tool call: **Read** puzzle.py

Result

```
1 import random
2
3 DIRECTIONS = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}
4 OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}
5
6
7 class Puzzle:
8     def __init__(self, size=4):
9         self.size = size
10        self.board = self.make_board()
11
12    def make_board(self):
13        # Start solved and scramble with legal blank moves, so every board is reachable.
14        tiles = list(range(1, self.size * self.size)) + [0]
15        self.board = [tiles[r * self.size:(r + 1) * self.size] for r in range(self.size)]
16        while self.solved():
17            self.scramble(self.size * self.size * 20)
18        return self.board
19
20    def scramble(self, steps):
21        last = None
22        for _ in range(steps):
23            options = [d for d in DIRECTIONS if d != OPPOSITE.get(last) and
24self.can_move(d)]
25            last = random.choice(options)
26            self.move(last)
27
28    def can_move(self, direction):
29        r, c = self.blank_pos()
30        dr, dc = DIRECTIONS[direction]
31        return 0 <= r + dr < self.size and 0 <= c + dc < self.size
32
33    def blank_pos(self):
```

```

33         for r in range(self.size):
34             for c in range(self.size):
35                 if self.board[r][c] == 0:
36                     return r, c
37
38     def move(self, direction):
39         if not self.can_move(direction):
40             return False
41         r, c = self.blank_pos()
42         dr, dc = DIRECTIONS[direction]
43         nr, nc = r + dr, c + dc
44         self.board[r][c], self.board[nr][nc] = self.board[nr][nc], self.board[r][c]
45         return True
46
47     def solved(self):
48         return sum(self.board, []) == list(range(1, self.size * self.size)) + [0]
49

```

Tool call: **Edit** puzzle.py

Replace:

```

DIRECTIONS = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}
OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}

```

With:

```

# Each key slides a tile in that direction into the blank, so the offset is
# where that tile sits relative to the blank (W = the tile below moves up).
DIRECTIONS = {"w": (1, 0), "s": (-1, 0), "a": (0, 1), "d": (0, -1)}
DIRECTION_NAMES = {"w": "up", "s": "down", "a": "left", "d": "right"}
OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}

```

Result

The file puzzle.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: **Edit** puzzle.py

Replace:

```

def can_move(self, direction):
    r, c = self.blank_pos()

```

With:

```

def can_move(self, direction):
    if direction not in DIRECTIONS:
        return False
    r, c = self.blank_pos()

```

Result

The file puzzle.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: **Edit** puzzle.py

Replace:

```

def move(self, direction):
    if not self.can_move(direction):
        return False
    r, c = self.blank_pos()
    dr, dc = DIRECTIONS[direction]
    nr, nc = r + dr, c + dc
    self.board[r][c], self.board[nr][nc] = self.board[nr][nc], self.board[r][c]
    return True

```

With:

```
def move(self, direction):
    """Slide a tile into the blank. Return the moved tile's number, or None if no tile moved."""
    if not self.can_move(direction):
        return None
    r, c = self.blank_pos()
    dr, dc = DIRECTIONS[direction]
    nr, nc = r + dr, c + dc
    tile = self.board[nr][nc]
    self.board[r][c], self.board[nr][nc] = tile, 0
    return tile
```

Result

The file puzzle.py has been updated successfully. (file state is current in your context – no need to Read it back)

Now the game side: exact command matching, a success message only after a real move, and handling for Ctrl+C or end of input.

Tool call: **Edit** game.py

Replace:

```
import time
from puzzle import Puzzle

SIZES = (3, 4, 5)
DEFAULT_SIZE = 4
```

With:

```
import time
from puzzle import DIRECTION_NAMES, DIRECTIONS, Puzzle

SIZES = (3, 4, 5)
DEFAULT_SIZE = 4

def read(prompt):
    # Treat Ctrl+C / end of input as a request to quit instead of crashing.
    try:
        return input(prompt).strip().lower()
    except (EOFError, KeyboardInterrupt):
        print()
        return "q"
```

Result

The file game.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: **Edit** game.py

Replace:

```
choice = input(f"Board size 3, 4 or 5 [Enter = {self.size}, Q quits]: ").strip().lower()
```

With:

```
choice = read(f"Board size 3, 4 or 5 [Enter = {self.size}, Q quits]: ")
```

Result

The file game.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: **Edit** game.py

Replace:

```
if not self.puzzle.move(key):
    print("That move is not possible.")
    return False
self.moves += 1
```

With:

```
tile = self.puzzle.move(key)
if tile is None:
    print(f"No tile can move {DIRECTION_NAMES[key]} into the blank.")
    return False
print(f"Moved tile {tile} {DIRECTION_NAMES[key]}.")
self.moves += 1
```

Result

The file game.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: **Edit** game.py

Replace:

```
print("Sliding Puzzle – W/A/S/D moves the tile into the blank. N new board, Q quits.")
```

With:

```
print("Sliding Puzzle – W/A/S/D slides a tile up/left/down/right into the blank.")
print("N starts a new board, Q quits.")
```

Result

The file game.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: **Edit** game.py

Replace:

```
key = input("> ").strip().lower()
```

With:

```
key = read("> ")
```

Result

The file game.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: **Edit** game.py

Replace:

```
if key not in "wasd":
    print("Use W/A/S/D, N or Q.")
```

With:

```
if key not in DIRECTIONS:
    print(f"Unknown command {key!r}. Use W/A/S/D, N or Q.")
```

Result

The file game.py has been updated successfully. (file state is current in your context – no need to Read it back)

Now the Task 4 checks, plus all the earlier test scripts as a regression run:

Tool call: **Write** scratchpad\task4_check.py

```
"""Task 4 checks: tile-moving semantics, success feedback only on real moves, invalid input."""
import builtins
import contextlib
import copy
import io
import sys

sys.path.insert(0, sys.argv[1])
from game import SlidingPuzzle

def make_game(board):
    g = SlidingPuzzle()
    g.new_board(len(board))
    g.puzzle.board = copy.deepcopy(board)
    return g

def captured(fn, *args):
    out = io.StringIO()
    with contextlib.redirect_stdout(out):
        result = fn(*args)
    return result, out.getvalue()

# Blank in the centre of a 3x3: every key moves the expected neighbour INTO the blank.
centre = [[1, 2, 3], [4, 0, 5], [6, 7, 8]]
expected = {"w": 7, "s": 2, "a": 5, "d": 4} # W: tile below moves up, etc.
for key, tile in expected.items():
    g = make_game(centre)
    ok, out = captured(g.slide, key)
    assert ok and g.moves == 1 and g.puzzle.board[1][1] == tile, (key, g.puzzle.board)
    assert f"Moved tile {tile}" in out, out
print("PASS W/A/S/D slide the correct tile into the blank, with success feedback")

# Blank in the bottom-right corner: W and A have no tile to slide in.
corner = [[1, 2, 3], [4, 5, 6], [8, 7, 0]]
for key in "wa":
    g = make_game(corner)
    ok, out = captured(g.slide, key)
    assert not ok and g.moves == 0 and g.total_moves == 0 and g.puzzle.board == corner
    assert "Moved" not in out and "No tile can move" in out, out
print("PASS impossible moves: no board change, no move count, no success message")

# Junk commands through run(): none may crash, move a tile, or count a move.
junk = ["", " ", "wa", "wasd", "x", "as", "up", "1", "ww", "?"]
g = SlidingPuzzle()
keys = iter(["3"] + junk + ["q"])
builtins.input = lambda prompt="": next(keys)
_, out = captured(g.run)
assert g.moves == 0 and g.total_moves == 0
assert out.count("Unknown command") == len(junk) and "Moved" not in out
print(f"PASS {len(junk)} invalid commands rejected without crashing or counting")

# Upper-case keys still work (input is case-insensitive).
g = make_game(centre)
keys = iter(["W", "q"])
builtins.input = lambda prompt="": next(keys)
g.puzzle.board = copy.deepcopy(centre)
g.run.__func__ # run() asks size first; drive the loop pieces directly instead
ok, _ = captured(g.slide, "W".lower())
assert ok and g.moves == 1
print("PASS upper-case keys accepted")

# End of input / Ctrl+C quits cleanly instead of crashing.
for exc in (EOFError, KeyboardInterrupt):
    def boom(prompt=""):
        raise exc
    builtins.input = boom
    g = SlidingPuzzle()
```

```
_, out = captured(g.run)
assert "Session:" in out
print("PASS EOF and Ctrl+C quit cleanly")
```

Result

File created successfully at: scratchpad\task4_check.py (file state is current in your context – no need to Read it back)

Tool call: **Bash** Run Task 4 checks plus regression checks

```
S="scratchpad"
python "$S/task4_check.py" . 2>&1 | grep -E "PASS|Error|Traceback|line [0-9]"
echo "--- regression"
python "$S/task3_check.py" . 2>&1 | grep -E "PASS|Error|Traceback|line [0-9]"
python "$S/parity_check.py" .
```

Result

```
PASS W/A/S/D slide the correct tile into the blank, with success feedback
PASS impossible moves: no board change, no move count, no success message
PASS 10 invalid commands rejected without crashing or counting
PASS upper-case keys accepted
PASS EOF and Ctrl+C quit cleanly
--- regression
PASS 3x3, 4x4, 5x5 boards well formed
PASS size prompt validates input
PASS timer runs and freezes identically for all sizes
PASS new board resets per-board values only; session values kept
PASS full run: solved 3x3 in 12 moves, post-solve moves rejected, switched to 4x4, quit
3x3: 0/1000 unsolvable, 0/1000 start already solved

4x4: 0/1000 unsolvable, 0/1000 start already solved

5x5: 0/1000 unsolvable, 0/1000 start already solved
```

All pass. My upper-case test in that script was weak: it lower-cased the key itself before calling the game. So I'm checking upper case, along with the old crash inputs, through the real `main.py`:

Tool call: **Bash** Play the real game with empty, junk, and upper-case input

```
printf '3\nnwa\nW\nD\nA\nq\n' | python main.py
```

Result

Sliding Puzzle  W/A/S/D slides a tile up/left/down/right into the blank.

N starts a new board, Q quits.

Board size 3, 4 or 5 [Enter = 4, Q quits]:

```
5 2
```

```
3 6 1
```

```
4 7 8
```

Size: 3x3 Moves: 0 Time: 0 s

> Unknown command '.'. Use W/A/S/D, N or Q.

```
5 2
```

```
3 6 1
```

```

4 7 8

Size: 3x3 Moves: 0 Time: 0 s

> Unknown command 'wa'. Use W/A/S/D, N or Q.


5 2
3 6 1
4 7 8

Size: 3x3 Moves: 0 Time: 0 s

> Moved tile 3 up.


3 5 2
6 1
4 7 8

Size: 3x3 Moves: 1 Time: 0 s

> No tile can move right into the blank.


3 5 2
6 1
4 7 8

Size: 3x3 Moves: 1 Time: 0 s

> Moved tile 6 left.


3 5 2
6 1
4 7 8

Size: 3x3 Moves: 2 Time: 0 s

> Quit without solving.

Session: 1 board(s), 0 solved, 2 total moves, 0 s.

```

Tool call: **Bash** Commit Task 4 and show history

```

git add game.py puzzle.py && git commit -q -F - <<'EOF'
Task 4: Report success only for real slides; reject invalid commands

- W/A/S/D now slide a tile up/left/down/right into the blank, matching the
  on-screen instructions (previously they moved the blank).
- Puzzle.move returns the moved tile, or None when no tile can move; the
  game prints "Moved tile N <dir>." and counts the move only in that case.
- Commands are matched exactly, so "", "wa", "wasd", etc. no longer pass the
  old substring check and crash with KeyError; they are rejected without
  changing state. Ctrl+C / end of input quit cleanly.

```

```

Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>
EOF
git log --oneline -5; git status --short

```

Result

```
warning: in the working copy of 'game.py', LF will be replaced by CRLF the next time Git touches it
0b26f02 Task 4: Report success only for real slides; reject invalid commands
f06e449 Task 3: Add 3x3/4x4/5x5 modes with consistent moves and timer
ee093a2 Task 2: Complete puzzle lifecycle
045a459 Task 1: Guarantee solvable starting boards
75858c1 Updated scenario title
?? BEFORE.mp4
?? __pycache__/
```

Task 4 is done and committed locally as `0b26f02`, which means all four tasks are complete. Nothing has been pushed yet.

What changed: - **Keys move the tile:** W/A/S/D now slide a tile up, left, down or right into the blank, as the title line says. Before, they moved the blank. The direction table is in [puzzle.py](#). - **Success message only after a real slide:** `Puzzle.move` now returns the number of the tile that moved, or nothing if no tile could move. The game prints "Moved tile 7 up." and adds to the move count only in that case. - **Impossible moves:** the game prints "No tile can move right into the blank." and nothing changes. - **Invalid commands:** commands must now match exactly. An empty line, `wa`, `wasd` and similar inputs used to slip past the old check and crash with `KeyError`; now they print "Unknown command" and change nothing. - **Ctrl+C or end of input** quits cleanly with the session summary instead of crashing.

Tests: - All four keys moved the correct tile into the blank, with the success message. - Impossible edge moves left the board, the move count and the session total unchanged, with no success message. - Ten junk commands were all rejected without crashing or counting. - Ctrl+C and end of input both quit cleanly. - The Task 1–3 checks still pass, including 0 unsolvable boards out of 1,000 at every size. - A real piped run of `main.py` behaved correctly, including upper-case keys.

What's left for you: 1. **Record the 10-second "after" video.** A 3x3 board is quickest. Show a few moves with the "Moved tile..." messages and one impossible move, then type something like `wa` and press N to switch size. If you can solve a 3x3, that also shows the "Solved" screen and moves being refused afterwards. 2. **Push.** I haven't pushed. Say the word and I'll push the four commits to `origin/main` (pranav-prabhakar-dev/22_sliding_puzzle), or you can run: `bash git push origin main` 3. **Export this chat** as a doc or PDF. 4. **Upload to your lab repo's Lab-4 folder:** both videos, the code and the chat export. `BEFORE.mp4` is still untracked here, and I've kept it out of every commit.

In the piped test output, the dash in the title line showed as `◆`. That character comes from the original code, and it's an encoding issue when output is piped; the game behaves the same either way. I haven't checked how it looks when the game runs directly in a terminal, so check it before recording. I can replace it with a plain `-` if you prefer.

USER - PROMPT 8

replace with `-` if that doesn't cause issues on any terminal

ASSISTANT - CLAUDE OPUS 5.5 (CLAUDE CODE)

A plain `-` is ASCII, so it shows the same on every terminal and encoding. First I'll check for any other non-ASCII characters in the code:

Tool call: **Grep** `[\x00-\x7F]`

Result

```
game.py:96:      print("Sliding Puzzle – W/A/S/D slides a tile up/left/down/right into the  
blank.")
```

Tool call: **Edit** game.py

Replace:

```
Sliding Puzzle – W/A/S/D
```

With:

```
Sliding Puzzle - W/A/S/D
```

Result

The file game.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: **Bash** Confirm no non-ASCII chars remain and check banner

```
LC_ALL=C grep -nP '[^\x00-\x7F]' *.py || echo "no non-ASCII left"; printf 'q\n' | python main.py | head  
-1
```

Result

```
no non-ASCII left  
Sliding Puzzle - W/A/S/D slides a tile up/left/down/right into the blank.
```

The title now prints cleanly. This change is cosmetic and doesn't belong to any one task, so I'll commit it separately to keep the four task commits clean:

Tool call: **Bash** Commit the hyphen change

```
git add game.py && git commit -q -F - <<'EOF'  
Use an ASCII hyphen in the title line
```

The em dash rendered as a replacement character on terminals and pipes that are not UTF-8. The source is now pure ASCII.

```
Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>  
EOF  
git log --oneline -6
```

Result

```
warning: in the working copy of 'game.py', LF will be replaced by CRLF the next time Git touches it  
83fcb47 Use an ASCII hyphen in the title line  
0b26f02 Task 4: Report success only for real slides; reject invalid commands  
f06e449 Task 3: Add 3x3/4x4/5x5 modes with consistent moves and timer  
ee093a2 Task 2: Complete puzzle lifecycle  
045a459 Task 1: Guarantee solvable starting boards  
75858c1 Updated scenario title
```

I replaced the – in the title line with a plain -. It now prints as "Sliding Puzzle - W/A/S/D slides a tile...". The game's code is now all plain ASCII, so it can't come out garbled on any terminal, encoding or piped output. The title line was the only non-ASCII character in the code.

I committed it on its own as 83fcb47, so the four task commits still contain only their own task.

Still to do: the "after" video, the push, the chat export and the Lab-4 upload. Tell me when to push.

USER - PROMPT 9

So, I can now record the "After" video?

ASSISTANT - CLAUDE OPUS 5.5 (CLAUDE CODE)

Yes, all four tasks are done and committed. Run it from a terminal in this folder:

```
python main.py
```

A sequence that fits in about 10 seconds: 1. **Pick a size:** type `3` and press Enter. This shows the new size choice. 2. **Make a real move:** press a key that works, such as `w`. You'll see "Moved tile 7 up." and Moves goes to 1. 3. **Try an impossible move:** you'll see "No tile can move ... into the blank." and Moves doesn't change. Which key is impossible depends on where the blank is: - blank on the bottom row: `w` - blank on the top row: `s` - blank in the right column: `a` - blank in the left column: `d` 4. **Try an invalid command:** press Enter on an empty line, or type `wa`. This crashed before and now shows "Unknown command". If your "before" video showed that crash, this is the clearest before-and-after contrast. 5. **Start a new board:** type `n`, then `5`. A 5x5 board appears with Moves and Time reset. 6. **Quit:** type `q`. This shows the session summary.

Tips: - Start recording once the size prompt is on screen, so the 10 seconds aren't spent typing the command. - Solving a board, which shows the "Solved" message and moves being refused afterwards, won't realistically fit in 10 seconds. That behaviour is already covered by the tests above, so leave it out. - The fix for unsolvable boards (Task 1) can't be seen in a short clip. The 0-out-of-1,000 check results earlier in this chat are your evidence for it.

When you've recorded it, tell me and I'll push the five commits to `origin/main`.

USER - PROMPT 10

push to main

ASSISTANT - CLAUDE OPUS 5.5 (CLAUDE CODE)

Tool call: **Bash** Push commits to origin main and confirm

```
git status --short; git push origin main 2>&1; git log --oneline origin/main -6
```

Result

```
?? __pycache__/  
bash: line 1: /dev/tty: No such device or address  
error: failed to execute prompt script (exit code 1)  
fatal: could not read Username for 'https://github.com': No such file or directory  
75858c1 Updated scenario title  
56c88a9 Initial commit
```

The push failed because Git Bash here has no terminal to ask for your GitHub login. I'll try once from PowerShell, where Git Credential Manager may already have your login saved:

Tool call: **PowerShell** Push commits to origin main via PowerShell

```
git push origin main 2>&1; git log --oneline origin/main -6
```

Result

```
fatal: could not read Username for 'https://github.com': terminal prompts disabled

75858c1 Updated scenario title
56c88a9 Initial commit
```

I couldn't push. GitHub asked for your login, and my shell can't show a login prompt. I also can't enter credentials for you. Nothing was sent, so `origin/main` is still at `75858c1`.

Please run this in your own terminal, from this folder:

```
git push origin main
```

The first time, a GitHub sign-in window should open, through Git Credential Manager, which comes with Git for Windows. After that the login is saved, and I can push for you in future sessions.

What will be pushed (5 commits):

```
83fcb47 Use an ASCII hyphen in the title line
0b26f02 Task 4: Report success only for real slides; reject invalid commands
f06e449 Task 3: Add 3x3/4x4/5x5 modes with consistent moves and timer
ee093a2 Task 2: Complete puzzle lifecycle
045a459 Task 1: Guarantee solvable starting boards
```

Only `__pycache__` is left untracked, and it won't be pushed. `BEFORE.mp4` no longer shows as untracked, so it seems to have been moved out of this folder. Keep it with your "after" video for the `Lab-4` upload.

After pushing, you can confirm it worked with:

```
git log --oneline origin/main -1
```

It should show `83fcb47`.

USER - PROMPT 11

okay all done, now chat history

ASSISTANT - CLAUDE OPUS 5.5 (CLAUDE CODE)

Tool call: **ToolSearch**

```
{
  "query": "select:mcp_ccd_session_mgmt_export_transcript",
  "max_results": 1
}
```

Tool call: **Bash** Confirm pushed commit on origin/main

```
git log --oneline origin/main -1
```

Result

83fcb47 Use an ASCII hyphen in the title line